

LLM

Os grandes modelos de linguagem (LLMs) são uma categoria de modelos de deep learning treinados em imensas quantidades de dados, tornando-os capazes de entender e gerar linguagem natural e outros tipos de conteúdo para executar uma grande variedade de tarefas. Os LLMs são construídos em um tipo de arquitetura de redes neurais chamada de transformador, que se destaca no tratamento de sequências de palavras e na captura de padrões em texto.

Os LLMs funcionam como enormes máquinas de previsão estatística que preveem repetidamente a próxima palavra em uma sequência. Eles aprendem padrões em seus textos e geram uma linguagem que segue esses padrões.

Os LLMs representam um grande salto na forma como os seres humanos interagem com a tecnologia porque são o primeiro sistema de IA que pode lidar com linguagem humana não estruturada em escala, permitindo uma comunicação natural com máquinas. Enquanto os mecanismos de pesquisa tradicionais e outros sistemas programados usavam algoritmos para corresponder palavras-chave, os LLMs capturam contexto, nuances e raciocínio mais profundos. Uma vez treinados, os LLMs podem se adaptar a muitas aplicações que envolvem interpretação de texto, como resumir um artigo, depurar código ou redigir uma cláusula legal. Quando recebem recursos de agentes, os LLMs podem executar, com diferentes graus de autonomia, várias tarefas que, de outra forma, seriam executadas por seres humanos.

Os LLMs são o apogeu de décadas de progresso no processamento de linguagem natural (NLP) e na pesquisa de aprendizado de máquina, e seu desenvolvimento é em grande parte responsável pela explosão de avanços em inteligência artificial no final das décadas de 2010 e 2020. LLMs populares se tornaram nomes conhecidos, trazendo a IA generativa para a vanguarda do interesse público. Os LLMs também são amplamente usados em empresas, com organizações investindo pesadamente em inúmeras funções de negócios e casos de uso.

Os LLMs são facilmente acessíveis ao público por meio de interfaces como o Claude da Anthropic, o ChatGPT da Open IA, o Copilot da Microsoft, os modelos Llama da Meta e o assistente Gemini do Google, juntamente com seus modelos BERT e PaLM. A IBM mantém uma série de modelos Granite no watsonx.ai, que se tornou a espinha dorsal da IA generativa de outros produtos da IBM, como o watsonx Assistant e o watsonx Orchestrate.

ré-treinamento de grandes modelos de linguagem

O treinamento começa com uma enorme quantidade de dados: bilhões ou trilhões de palavras de livros, artigos, sites, código e outras fontes de texto. Cientistas de dados supervisionam a limpeza e o pré-processamento para remover erros, duplicações e conteúdo indesejável.

Esse texto é dividido em unidades menores e legíveis por máquina chamadas "tokens", durante um processo de "tokenização". Tokens são unidades menores,

como palavras, subpalavras ou caracteres. Isso padroniza a linguagem para que palavras raras e novas possam ser tratadas de forma consistente.

Inicialmente, os LLMs são treinados com [aprendizado autossupervisionado](#), uma técnica de [aprendizado de máquina](#) que usa dados não rotulados para o [aprendizado supervisionado](#). O aprendizado autossupervisionado não requer conjuntos de dados rotulados, mas está intimamente relacionado ao aprendizado supervisionado, pois otimiza o desempenho em relação a uma "verdade absoluta". No aprendizado autossupervisionado, as tarefas são projetadas de forma que a "verdade absoluta" possa ser inferida a partir de dados não rotulados. Em vez de ser informado sobre qual é a "saída correta" para cada entrada, como no aprendizado supervisionado, o modelo tenta encontrar padrões, estruturas ou relacionamentos nos dados por conta própria.

Autoatenção

O modelo passa os tokens por uma rede de [transformadores](#). Os modelos de transformadores, lançados em 2017, são úteis devido ao seu [mecanismo de autoatenção](#), que lhes permite "prestar atenção a" diferentes tokens em diferentes momentos. Essa técnica é a peça central do transformador e sua principal inovação. A autoatenção é útil em parte porque permite que o [modelo de IA](#) calcule as relações e dependências entre os tokens, especialmente aqueles que estão distantes uns dos outros no texto. As arquiteturas de transformação também permitem a paralelização, tornando o processo muito mais eficiente do que os métodos anteriores. Essas qualidades permitiram que os LLMs lidassem com [conjuntos de dados](#) sem precedentes.

Depois que o texto é dividido em tokens, cada token é mapeado para um vetor de números chamado de [embedding](#). As redes neurais consistem em camadas de neurônios artificiais, onde cada neurônio realiza uma operação matemática. Os transformadores consistem em muitas dessas camadas, e em cada uma delas, as embeddings são ligeiramente ajustadas, tornando-se representações contextuais mais ricas de camada para camada.

O objetivo nesse processo é que o modelo aprenda associações semânticas entre palavras, de modo que palavras como "latido" e "cão" apareçam mais próximas no

espaço vetorial em um ensaio sobre cães do que "latido" e "árvore", com base em as palavras relacionadas a cães no ensaio. Os transformadores também adicionam [codificações posicionais](#), que dão a cada token informações sobre seu lugar na sequência.

Para calcular a atenção, cada embedding é projetada em três vetores distintos usando matrizes de peso aprendidas: uma consulta, uma chave e um valor. A consulta representa o que um determinado token está "procurando", a chave representa as informações que cada token contém e o valor "retorna" as informações de cada vetor de chave, dimensionadas de acordo com seu respectivos peso de atenção.

As pontuações de alinhamento são, então, calculadas como a similaridade entre consultas e chaves. Essas pontuações, uma vez normalizadas em pesos de atenção, determinam o quanto de cada vetor de valor flui para a representação do token atual. Esse processo permite que o modelo se concentre de forma flexível no contexto relevante, ignorando tokens menos importantes (como "árvore").

A [autoatenção](#), portanto, cria conexões "ponderadas" entre todos os tokens de forma mais eficiente do que as arquiteturas anteriores conseguiam. O modelo atribui pesos a cada relacionamento entre os tokens. Os LLMs podem ter bilhões ou trilhões desses pesos, que são um tipo de [parâmetro do LLM](#), as variáveis de configuração interna de um modelo de aprendizado de máquina que controlam como ele processa dados e faz previsões. O número de parâmetros refere-se a quantas dessas variáveis existem em um modelo, com alguns LLMs contendo bilhões de parâmetros. Os chamados [pequenos modelos de linguagem](#) são menores em escala e escopo, com comparativamente poucos parâmetros, tornando-os adequados para implementação em dispositivos menores ou em ambientes com recursos restritos.

Durante o treinamento, o modelo faz previsões em milhões de exemplos extraídos de seus [dados de treinamento](#), e uma [função de perda](#) quantifica o erro de cada previsão. Por meio de um ciclo iterativo de fazer previsões e, em seguida, atualizar os pesos do modelo por meio de [retropropagação](#) e [gradiente descendente](#), o

modelo "aprende" os pesos nas camadas que produzem os vetores de consultas, chaves e valores.

Quando esses pesos estiverem suficientemente otimizados, eles poderão receber a [embedding de vetores](#) original de qualquer token e produzir vetores de consultas, chaves e valores para ele que, ao interagir com os vetores gerados para todos os outros tokens, produzirão pontuações de alinhamento “melhores” que, por sua vez, resultam em pesos de atenção que ajudam o modelo a produzir melhores saídas. O resultado final é um modelo que aprendeu padrões de gramática, fatos, estruturas de raciocínio, estilos de escrita e muito mais.

Ajuste fino de grandes modelos de linguagem

Após o treinamento (ou no contexto de treinamento adicional, "pré-treinamento"), os LLMs podem ser ajustados para torná-los mais úteis em determinados contextos. Por exemplo, um modelo fundamental treinado em um grande conjunto de dados de conhecimento geral pode ser ajustado em um corpus de perguntas e respostas jurídicas para criar um [chatbot](#) para a área jurídica.

Ajuste fino supervisionado

O ajuste fino geralmente ocorre em um contexto supervisionado com um conjunto de dados rotulado muito menor. O modelo atualiza seus pesos para melhor corresponder à nova verdade absoluta (nesse caso, dados rotulados).

Enquanto o pré-treinamento visa dar ao modelo um amplo conhecimento geral, o ajuste fino adapta um modelo de propósito geral a tarefas específicas, como sumarização, classificação ou suporte ao cliente. Essas *adaptações funcionais* representam novos tipos de tarefas. O ajuste fino supervisionado produz saídas mais próximas dos exemplos fornecidos por seres humanos, exigindo muito menos recursos do que o treinamento a partir do zero.

O ajuste fino supervisionado também é útil para a [personalização específica do domínio](#), como treinar um modelo com documentos médicos para que ele possa responder a perguntas relacionadas a saúde.

Aprendizado por reforço a partir do feedback humano

Para refinar ainda mais os modelos, os cientistas de dados costumam usar o [aprendizado por reforço a partir do feedback humano](#) (RLHF), uma forma de ajuste fino em que os humanos classificam as saídas do modelo, e o modelo é treinado para preferir as saídas que os seres humanos classificam como mais altas. O RLHF é frequentemente usado em alinhamento, um processo que consiste em tornar as saídas do LLM úteis, seguras e consistentes com os valores humanos.

O RLHF também é particularmente útil para *alinhamento estilístico*, onde um LLM pode ser ajustado para responder de uma maneira mais casual, bem-humorada ou consistente com a marca. O alinhamento estilístico envolve treinamento para os mesmos tipos de tarefas, mas produzindo saídas em um estilo específico.

Modelos de raciocínio

O ajuste fino puramente supervisionado ensina um modelo a imitar exemplos, mas não necessariamente incentiva um melhor raciocínio, que envolve processos abstratos e de várias etapas. Essas tarefas nem sempre têm dados rotulados em abundância; portanto, o [aprendizado por reforço](#) é frequentemente usado na criação de [modelos de raciocínio](#), LLMs que receberam ajuste fino para dividir problemas complexos em etapas menores, geralmente chamadas de "traços de raciocínio", antes de gerarem uma saída final. Meios cada vez mais sofisticados de treinar modelos oferecem raciocínio em cadeia de pensamento e outras estratégias de tomada de decisão multietapas.

Ajuste de instruções

Outra forma de [personalização de LLMs](#) é o [ajuste de instruções](#), um processo projetado especificamente para melhorar a capacidade de um modelo de seguir instruções humanas. As amostras de entradas em um conjunto de dados consistem inteiramente em tarefas que se assemelham às solicitações que os usuários podem fazer em seus prompts; as saídas demonstram respostas desejáveis a essas solicitações. Como os LLMs pré-treinados não são inerentemente otimizados para seguir instruções ou objetivos de conversa, o ajuste de instruções é usado para alinhar melhor o modelo com a intenção do usuário.

Uso de grandes modelos de linguagem

Uma vez treinados, os grandes modelos de linguagem funcionam respondendo a prompts, tokenizando o prompt, convertendo-o em embeddings e usando seu transformador para gerar texto, um token de cada vez, calculando as probabilidades

de todos os possíveis próximos tokens e produzindo o mais provável. Esse processo, chamado de [inferência](#), é repetido até que a saída esteja concluída. O modelo não “sabe” a resposta final com antecedência; ele usa todas as relações estatísticas que aprendeu no treinamento para prever um token de cada vez, dando o seu melhor palpite em cada etapa.

A maneira mais fácil e rápida de obter conhecimento específico de um LLM de uso geral é por meio da [engenharia de prompts](#), que não requer treinamento adicional. Os usuários podem modificar prompts de todas as maneiras. Por exemplo, um prompt como "responda na voz de um profissional de saúde treinado" pode gerar resultados mais relevantes (observe que os LLMs não são recomendados para serem usados para aconselhamento médico!).

Os LLMs têm outras estratégias para controlar suas saídas, como a [temperatura do LLM](#), que controla a aleatoriedade do texto gerado pelos LLMs durante a inferência, ou a amostragem top-k/top-p, que limita o conjunto de tokens considerados aos mais prováveis, equilibrando criatividade e coerência.

A [janela de contexto](#) é o número máximo de tokens que um modelo pode "ver" e usar de uma só vez ao gerar texto. Os primeiros LLMs tinham janelas curtas, mas os LLMs mais recentes têm centenas de milhares de tokens em suas janelas de contexto, permitindo casos de uso como resumir documentos de pesquisa inteiros, realizar assistência de código em grandes bases de código e manter conversas longas e contínuas com os usuários.

A [geração aumentada de recuperação](#) (RAG) é um método para conectar um [modelo pré-treinado](#) a bases de conhecimento externas, permitindo que forneça respostas mais relevantes em um nível mais elevado de precisão. As informações recuperadas são passadas para a janela de contexto do modelo, para que o modelo possa usá-las ao gerar respostas, sem precisar de retreinamento. Por exemplo, ao conectar um LLM a um banco de dados dinâmico de serviços meteorológicos, um LLM pode recuperar informações para um usuário sobre o boletim meteorológico daquele dia.

Implementação de LLMs

Criar um LLM a partir do zero é um processo complexo e que consome muitos recursos. Os LLMs mais populares são o resultado de imensas quantidades de dados, [GPUs](#), energia e conhecimento humano, e é por isso que a maioria é criada e mantida por grandes empresas de tecnologia com recursos expansivos.

No entanto, muitos desses modelos são acessíveis a todos os desenvolvedores por meio de APIs. Os desenvolvedores podem usar [modelos pré-treinados](#) para construir chatbots, sistemas de recuperação de conhecimento, ferramentas de [automação](#) e muito mais. Para maior controle sobre os dados e a personalização, muitos modelos de código aberto podem ser implementados no local ou na nuvem. Github, [Hugging Face](#), Kaggle e outras plataformas tornam o desenvolvimento de IA acessível a todos.

Os desenvolvedores podem usar LLMs como base para todos os tipos de aplicações de IA. Um dos desenvolvimentos mais interessantes na IA é o sistema agêntico. Os agentes de IA não apenas pensam; eles fazem. Por si só, os LLMs simplesmente geram texto com base no contexto, mas podem ser integrados com memória, [APIs](#), lógica de decisão e outros sistemas externos para executar tarefas específicas, como reservar um voo ou pilotar um veículo autônomo.

Casos de uso de grandes modelos de linguagem

Os LLMs estão redefinindo os processos de negócios e provaram sua versatilidade em uma infinidade de casos de uso em muitos setores.

Geração de texto: os LLMs podem realizar todos os tipos de tarefas de criação de conteúdo, como redigir e-mails, posts de blogs ou memorandos legais em resposta a prompts.

Sumarização de texto: os LLMs podem resumir artigos longos, notícias, relatórios de pesquisa, documentação corporativa e históricos de clientes em textos completos adaptados ao tamanho para um formato e estilo de saída desejados.

Assistentes de IA: chatbots impulsionados por [IA conversacional](#) podem realizar [respostas a perguntas](#) e fornecer informações detalhadas como parte de uma solução integrada de atenção ao cliente em tempo real.

Geração de código: as plataformas de assistência de código ajudam os desenvolvedores a criar aplicações, encontrar erros no código e descobrir problemas de segurança em várias linguagens de programação, até mesmo na tradução entre elas.

Análise de sentimento: o tom do cliente é analisado para entender melhor o feedback do cliente em escala.

Tradução de idiomas: a tradução automatizada oferece uma cobertura mais ampla para organizações em todos os idiomas e regiões geográficas, com traduções fluentes e recursos multilíngues.

Raciocínio: os LLMs podem resolver problemas matemáticos, planejar processos multietapas e explicar conceitos complexos em termos mais simples.

Avaliação de LLMs

Os LLMs são ferramentas poderosas, mas vêm com várias limitações. Uma grande preocupação é a precisão. Durante as **alucinações**, o modelo gera informações falsas ou enganosas, embora pareçam plausíveis. Os LLMs também podem refletir e amplificar **vieses** presentes em seus dados de treinamento, produzindo saídas injustas ou ofensivas. Além disso, suas demandas de recursos são significativas: treinar e executar LLMs exige grandes quantidades de poder computacional e energia, levantando preocupações ambientais e de custos.

Os profissionais podem mitigar esses aspectos negativos dos LLMs por meio de uma **governança de IA**, os processos, normas e proteções que ajudam a garantir que os sistemas e ferramentas de IA sejam seguros e éticos. Uma parte-chave da governança envolve a avaliação de modelos em relação a benchmarks. Os **benchmarks de LLMs** fornecem pontuações quantitativas, facilitando a comparação de modelos. Como os LLMs são sistemas de uso geral capazes de realizar uma ampla variedade de tarefas, sua avaliação requer várias dimensões em vez de um único benchmark. Pesquisadores e profissionais analisam qualidades como precisão, eficiência, segurança, imparcialidade e robustez para determinar o desempenho de um modelo.

Os LLMs também são avaliados com base no alinhamento e na segurança, com técnicas como red-teaming, em que avaliadores tentam intencionalmente fazer com que o modelo produza respostas inseguras ou com viés para expor pontos fracos. Avaliações de imparcialidade e viés podem ajudar os profissionais a evitar que os LLMs reproduzam estereótipos prejudiciais ou desinformação.

Os LLMs também são comumente avaliados com base na eficiência. Velocidade, consumo de energia, taxa de transferência de tokens, tamanho da memória e capacidade de lidar com longas janelas de contexto são algumas das métricas comuns usadas para avaliar a eficiência dos LLMs para chegar às saídas.

Uma breve história dos LLMs

A história dos LLMs remonta aos primórdios da computação e do processamento de linguagem natural, quando os pesquisadores usavam sistemas baseados em regras e métodos estatísticos para modelar o texto. Essas abordagens iniciais conseguiam capturar padrões de palavras locais, mas não conseguiam entender dependências de longo alcance ou semânticas mais profundas.

Uma grande mudança ocorreu na década de 2010, com o surgimento das redes neurais, com embeddings como Word2Vec e GloVe, que representavam palavras como vetores no espaço contínuo, permitindo que modelos aprendessem relações semânticas. Modelos de sequência como redes neurais recorrentes (RNNs) e redes de memória de curto longo prazo (LSTM) surgiram para lidar melhor com dados sequenciais.

Em 2017, Vaswani *et al.* introduziram a arquitetura de transformação [codificador-decodificador](#) no artigo de referência "Attention Is All You Need".[1] Os transformadores tornaram possível treinar modelos em grandes conjuntos de dados, marcando o início da era moderna dos LLMs. O BERT (2018) do Google, um transformador somente de codificador, demonstrou o poder dos transformadores para entender o idioma, enquanto a série de transformadores pré-treinados generativos (GPT) da OpenAI, baseada em uma variante somente do decodificador, mostrou como o pré-treinamento generativo em texto em escala de internet poderia produzir uma geração de idiomas notavelmente fluente. Mais ou menos na mesma época, modelos codificadores-decodificadores como o T5 do Google e o BART do

Facebook demonstraram os pontos fortes do projeto completo de sequência para sequência para tarefas como tradução e sumarização. O GPT-2 (2019) atraiu a atenção por sua capacidade de gerar parágrafos coerentes, enquanto o GPT-3 (2020), com 175 bilhões de parâmetros, consolidou os LLMs como uma força transformadora na IA.

Além disso, novas arquiteturas estão desafiando a popularidade dos transformadores em LLMs. Os [modelos Mamba](#) funcionam usando um modelo de espaço de estado com atualizações seletivas que filtram e combinam com eficiência as informações passadas, permitindo capturar dependências de longo alcance. Os [LLMs de difusão](#) começam com ruído aleatório e gradualmente o eliminam, guiados por um modelo aprendido, até que surja um texto coerente. Ambas as arquiteturas podem ser muito mais eficientes do que os transformadores.

FONTE: <https://www.ibm.com/br-pt/think/topics/large-language-models>